

Risk-Routed KV Cache Compression for Long-Context Inference

Anonymous submission

Abstract

Long-context inference keeps key/value states for every retained prompt token, so memory pressure and decoding latency grow with context length even when the model weights are fixed. Existing training-free KV-cache compression methods usually expose a fixed budget or a local salience rule, but long-context tasks differ in how costly a mistaken eviction is. This draft studies a risk-routed KV policy that starts from an aggressive retained-token budget and escalates only rows whose task family or compression-signal disagreement indicates reliability risk. The final paper will evaluate Qwen2.5-7B-Instruct on LongBench, RULER, and needle-in-a-haystack retrieval against full cache, SnapKV, H2O-compatible observed attention, StreamingLLM, and maintained kvpress baselines. The paper-facing result is currently unavailable: the full benchmark frontier is TBD_MAIN_FRONTIER_DELTA, peak-memory reduction is TBD_PEAK_MEMORY_REDUCTION, decoding-throughput gain is TBD_DECODE_THROUGHPUT_GAIN, and matched-budget quality is TBD_MATCHED_BUDGET_QUALITY.

Introduction

Autoregressive Transformers cache key/value states from previous tokens so each decoding step can attend to the retained context without recomputing the entire prefix (Vaswani et al. 2017). This cache is operationally useful but expensive: as prompts reach tens or hundreds of thousands of tokens, the cache can become a first-order memory and throughput bottleneck even when serving systems manage allocation efficiently (Shazeer 2019; Dao et al. 2022; Kwon et al. 2023). The core problem in this project is therefore not whether one can delete cache entries, but when a training-free policy can delete them without erasing evidence that a long-context task will later need.

Fixed-budget KV compression is a strong starting point. Heavy-hitter eviction, attention sinks, observation-window selection, layer-wise budgets, and adaptive per-head allocation have all shown that many tokens can be dropped or downweighted without immediately destroying generation quality (Zhang et al. 2023; Xiao et al. 2023; Li et al. 2024; Cai et al. 2024; Feng et al. 2024). Recent work also broadens the design space to quantization, semantic chunking, reconstruction-aware eviction, and shared compressed pools

(Liu et al. 2024, 2025; Wu et al. 2026; An et al. 2026; Patel and Joshi 2026). This crowded literature makes a generic “better salience score” thesis weak unless it is backed by direct, matched-budget evidence. It also makes single-number reporting risky. A method can look attractive when evaluated at one convenient budget, yet be operationally less useful if it spends too much cache on easy rows, fails on rows with rare but important evidence, or improves a benchmark average while hurting tasks that require exact retrieval. For a deployment-facing cache policy, the relevant object is a frontier over quality, realized KV cost, and latency, not a single retained-token percentage.

We instead frame the paper around a narrower operational question: can a training-free router improve the quality-versus-KV-budget frontier by spending more cache only on rows that appear risky before the answer is known? The planned method, Risk-routed KV, uses benchmark task family and disagreement among candidate KV presses as non-oracle reliability signals. It begins at a low retained-token budget and escalates to safer budgets for rows where an aggressive press is likely to remove task-critical evidence. This is a budget-allocation policy over existing presses, not a learned model and not an oracle that reads gold answers. The key hypothesis is therefore not that one press dominates every other press. It is that some prompts expose enough pre-answer risk information to justify heterogeneous cache budgets, and that this heterogeneity can be evaluated without peeking at task labels. That distinction matters because a training-free router should be easy to attach to existing inference stacks, but only if its benefit survives row-paired comparisons against fixed-budget and adaptive baselines.

The evidence contract is deliberately conservative. The final paper must report a cross-family frontier on LongBench (Bai et al. 2024), RULER (Hsieh et al. 2024), and an independent needle-in-a-haystack retrieval grid; it must also reproduce at least one strong baseline and measure peak memory and decoding throughput on a single H100. At the time of this draft update, a supervised full-tail run is live and nonterminal, and it has written only a partial prefix of the missing-method rows. Those partial rows are useful for run monitoring but are not paper-facing final evidence until the full tail is terminal, validated, and aggregated with the earlier incomplete frontier run. Accordingly, every outcome-dependent statement below is a result placeholder tied to

paper/RESULT_PLACEHOLDERS.md, not a claim.

The intended contributions, once the runs complete, are: a no-training risk-routed cache-budget policy; an exact accounting protocol for realized retained KV tokens and bytes; a real-benchmark frontier against full cache and prior KV-compression baselines; and a failure analysis of rows where extra KV budget does not recover quality. If the measured frontier does not support the policy, the final manuscript should keep the accounting protocol and negative analysis while softening or removing the method-superiority claim.

Related Work

Efficient attention and KV serving. Transformer self-attention creates the key/value states cached during autoregressive decoding (Vaswani et al. 2017). Multi-query attention reduces key/value duplication architecturally (Shazeer 2019), while FlashAttention and PagedAttention improve exact attention or memory management without making lossy token-retention decisions (Dao et al. 2022; Kwon et al. 2023). These systems motivate measuring memory and throughput, but they do not answer which evidence tokens a lossy compression policy can safely remove. They also establish an important evaluation boundary for this paper. Exact serving optimizations and lossy KV compression are complementary only when the compression method reports realized system cost honestly. If a press reduces nominal retained tokens but adds expensive scoring passes or breaks batching, the method may not improve the serving objective even if its benchmark score is acceptable. The final systems subset is therefore part of the claim, not an implementation afterthought.

Token eviction and streaming baselines. H2O treats high-attention heavy hitters as important cache entries (Zhang et al. 2023). StreamingLLM shows that attention sinks and recent tokens are special for stable long streams (Xiao et al. 2023). SnapKV uses an observation window before generation to identify useful prompt tokens (Li et al. 2024), and SAGE-KV further studies self-attention-guided one-shot eviction (Wang et al. 2025). These methods make attention-based eviction a strong baseline family; Risk-routed KV must compare against them rather than only against full cache or random dropping.

Adaptive allocation, quantization, and merging. PyramidKV and Ada-KV allocate budgets dynamically across layers or heads (Cai et al. 2024; Feng et al. 2024). KIVI attacks the same memory problem through asymmetric low-bit quantization rather than eviction (Liu et al. 2024). ChunkKV, SemantiCache, RocketKV, and ReST-KV show that chunk boundaries, staged filtering, merging, and reconstruction objectives are active alternatives (Behnam et al. 2025; Liu et al. 2025; Wu et al. 2026; An et al. 2026). This paper therefore scopes its method as a router over training-free retention budgets and reports realized KV cost, rather than claiming universal compression novelty.

Long-context benchmarks and failure analysis. LongBench provides real multitask long-context understanding rows, including QA, summarization, retrieval, few-shot,

synthetic, and code tasks (Bai et al. 2024). RULER shows that simple needle retrieval can overstate long-context ability and introduces controlled stress tasks beyond vanilla retrieval (Hsieh et al. 2024). Recent failure-focused work warns that cache compression can produce brittle task-specific failures (Chen et al. 2025; Bui et al. 2026). This motivates the paper’s evaluation shape: a frontier across independent benchmark families, paired row-level comparisons, and an explicit failure taxonomy. The three-family design is intentionally redundant. LongBench checks whether the method survives natural task variation; RULER stresses controlled operations that can be hidden inside aggregate long-context scores; and needle retrieval isolates positional survival under a transparent target. Agreement across these families would make the frontier easier to interpret, while disagreement would still be valuable because it would identify which long-context behaviors are not captured by a single benchmark average.

Background and Problem Definition

Let a prompt contain a sequence of tokens whose full-cache inference would retain all layer-wise key/value states. A compression method is assigned a retained-budget fraction such as 25%, 50%, 75%, or 100%. In this draft, budget fractions are paper-facing summaries; the artifact contract records realized retained tokens, estimated or measured KV bytes, and method-specific reason codes for every row. The main comparison is not a single fixed budget. It is the quality-versus-KV-budget frontier: for each method, the final analysis will compare benchmark quality against the mean realized KV budget and measured systems cost.

The accounting unit is a scored row, not a benchmark-level aggregate. For each row, the run contract records the input source identifier, tokenized prompt length, requested generation budget, selected press, retained-token count, realized retained fraction, answer text, score, and timing fields when available. For fixed-budget methods, the selected budget is known before routing. For Risk-routed KV, the selected budget is an output of the router, so the final frontier must use realized budget rather than the default budget printed in the method name. This prevents an adaptive method from being compared as if it had paid only for its cheapest path.

The target setting is training-free inference. The router may use prompt metadata, task family, candidate press behavior, and token-level quantities available before the gold answer is known. It may not use answer labels, exact-oracle lookup, benchmark-specific scoring feedback, or future task outcomes to choose a budget. This no-oracle rule is important because a method that spends extra KV only on rows it already knows it will miss would not be deployable. The same rule also governs analysis. Failure categories may be assigned after scoring, but they cannot be fed back into the budget choice for the row that produced them. If a future version uses learned predictors, that would become a different method with separate training data, validation splits, and leakage checks. This manuscript instead keeps the decision rule auditable and pre-answer.

The strongest possible result would show that Risk-routed KV is Pareto competitive: at matched or interpolated mean KV budget, it improves benchmark quality over fixed-budget baselines, or it preserves quality while reducing measured memory or increasing throughput. The current scaffold does not assert that result. The owner artifact for the final frontier is the full-run summary artifact, and its main numeric summary is `TBD_CI_FRONTIER_AREA`.

Risk-Routed KV Compression

Risk-routed KV is a thin training-free policy over existing KV presses. The default action is aggressive compression with a low retained-token budget. The router then applies two classes of reliability signals. First, task-family priors mark rows where aggressive cache deletion is expected to be riskier, such as multi-document QA, long summarization, controlled variable tracking, or deep-position retrieval. Second, row-level disagreement among candidate presses flags prompts whose token-importance profiles are unstable under the available salience views. When either signal crosses the configured risk threshold, the row escalates to a safer retained-token budget rather than changing the gold answer, prompt, or scorer.

The routing path has four deterministic stages. The loader first maps every benchmark row to a paper-facing family label and a source-row hash. The press probe then computes the token sets or retention scores available from the configured training-free presses, using only prompt-side information. The disagreement step compares candidate retained sets after normalizing them to the same nominal budget; high disagreement is interpreted as uncertainty about which evidence span is safe to drop. Finally, the budget selector chooses the default, middle, or safe retained budget and emits reason codes that can be audited independently of the answer score. These reason codes are not meant to prove correctness. They are there so a negative result can identify whether the router failed because the signal was uninformative, because the safer budget was still too small, or because the model did not use the preserved evidence.

The implementation plan uses NVIDIA kvpress as the primary maintained framework. Full cache is the no-press baseline. SnapKV, StreamingLLM, PyramidKV, and Ada-KV-style wrappers are planned through kvpress where supported. H2O is treated carefully: the local H2O reference clone lacks a confirmed top-level license in this workspace, so the draft names an H2O-compatible observed-attention path until reproduction and license status are resolved. The paper-facing H2O status is `TBD_H2O_STATUS`.

The policy is intentionally conservative about what counts as a baseline. Random eviction, dry-run accounting, and smoke rows can test code paths, but they cannot serve as the paper’s main comparison. For the paper-facing matrix, a baseline must run on the same source rows as Risk-routed KV, expose the same scoring fields, and be measured under the same model and decoding settings. If a published baseline cannot be reproduced with license and implementation provenance, the manuscript must name the substitution clearly rather than folding it into the original method name.

The router exposes its decision as an auditable budget trace rather than as a hidden postprocessor. For each row, the trace records the default press, the candidate safer press, the normalized overlap or disagreement signal used by the router, and the reason code that selected the final budget. This trace is not a paper-facing outcome by itself, but it lets the analysis separate two failure modes that would otherwise be merged in an aggregate score. One failure mode is signal failure: the trace shows no escalation even though a safer budget would have preserved task evidence. The other is budget failure: the trace escalates as intended, but the safer budget or press still does not preserve evidence the model can use. Keeping these cases separate is important for a training-free method because the repair action differs: a signal failure points to better pre-answer risk features, while a budget failure points to stronger presses, larger safe budgets, or a boundary where cache compression is not appropriate.

Each row must write the method label, benchmark family, source row ID, retained KV tokens, retained KV bytes or a validated proxy, realized budget fraction, escalation reason codes, answer text, score, latency, and memory telemetry when available. This accounting policy is what makes an adaptive policy comparable to a fixed-budget curve. A route that improves quality solely by spending more KV than the fixed frontier predicts will be reported as a boundary, not as a win. Conversely, a route that saves memory but loses too much task quality should be reported as a useful operating point only for scenarios where that loss is acceptable. The final text should therefore avoid unqualified superiority language. The supported claim will be selected after the row-level frontier is generated: a frontier improvement, a reliability-cost trade-off, or a negative diagnostic result.

Experimental Setup

Backbone and framework. The planned paper-facing model is Qwen2.5-7B-Instruct, using the project-local open-weight cache and the kvpress framework. The method is training-free: no trainable router, adapter, scorer head, or RL loop is planned. All large inference commands are constrained to the allocated H100 and must write raw row-level artifacts before any analysis claim is made.

Benchmarks. The planned benchmark families are LongBench, RULER, and needle-in-a-haystack retrieval. LongBench supplies real multitask long-context examples; RULER supplies controlled long-context stress tasks; the NIAH grid isolates retrieval survival by context length and needle position. The prepared source files and hashes are recorded in `experiments/BENCHMARK_PROVENANCE.json`. Prepared benchmark rows are not executed evidence; the live row owner artifacts are `TBD_NUMBER_OF_RUN_ROWS`. The sampling policy is designed to preserve semantic uniqueness. Rows may be filtered for feasibility, context length, and source availability, but a row cannot be duplicated with a new identifier to inflate the matrix. When pilot slices are used, they must remain labeled as pilot evidence until the full family and condition matrix is collected. This protects the paper from

Method family	Role	Budget policy	Evidence condition before paper-facing claim
Full cache	Upper-cost baseline	Retain all prompt KV	Completed rows in every benchmark family
SnapKV	Required published baseline	Fixed 25/50/75 cells	P-SnapKV
StreamingLLM	Required published baseline	Sink-plus-recent cells	P-Stream
H2O-compatible served attention	Required if authenticity passes	Heavy-attention cells	P-H2O
PyramidKV / Ada-KV	Maintained runner-up	Dynamic/adaptive cells	P-Strong
Risk-routed KV	Proposed policy	Low default with escalation	P-Router and rows

Table 1: Planned method roles and accounting gates. No result cell is paper-facing until its owner artifact has completed raw rows and a summary.

treating harness readiness as empirical evidence.

Baselines and frontier cells. The final matrix must include full cache; SnapKV fixed-budget cells; StreamingLLM fixed-budget cells; H2O or the documented H2O-compatible substitute; a strong maintained kvpress baseline such as PyramidKV; an adaptive runner-up such as Ada-KV; and Risk-routed KV plus routing ablations. The main result will compare quality at 25%, 50%, 75%, full-cache, and realized adaptive budgets. The full-cache score owner is `TBD_FULL_CACHE_BASELINE_SCORE`.

Metrics. Quality metrics follow the benchmark family: LongBench task metrics and macro averages, RULER exact or normalized task scores, and retrieval accuracy or answer containment for NIAH. Systems metrics include peak GPU memory, prefill throughput, decode throughput, end-to-end latency, and examples per second. Statistical support is planned as paired row-level comparisons and bootstrap confidence intervals. The final significance statement is `TBD_SIGNIFICANCE_RESULT`.

Execution protocol. Every paper-facing condition must be row-paired wherever the benchmark permits it: full cache, fixed-budget baselines, adaptive baselines, and Risk-routed KV should see the same prompt and scorer for a given source row. The run manifest must freeze the model identifier, decoding budget, selected benchmark split, method list, and stopping rules before the first expensive inference call. The runner must append progress before and after each trial and preserve failed rows rather than deleting them. If the supervised tail or a later aggregate stops before completing the required raw-row matrix, the manuscript must continue to use placeholders instead of extrapolating from partial prefixes or readiness checks.

Admission versus evidence. The run harness distinguishes row admission from paper-facing aggregation. Admission is a local integrity check: a row can be kept in raw artifacts when the source identity, method label, score parse, generation-cap status, retained-budget accounting, and telemetry fields are present. Paper-facing aggregation is stricter. It requires the planned method-source-row pairs to be complete, duplicate-safe across parent and continuation runs, and validated under the same source split before any table or prose backfill occurs. This separation lets the project keep negative, capped, or failed rows as evidence without

letting a live prefix become an accidental benchmark result. It also makes timeout recovery mechanical: a continuation can be judged by exact pair IDs rather than by trusting a status counter or a partial summary file.

Row validation and cap handling. The collection protocol treats each method-source-row pair as admissible only after duplicate checks, source-family accounting, score parsing, retained-KV accounting, memory telemetry, and generation-cap flags have been written for that pair. Generation caps are not automatically failures: a capped row is valid only if the benchmark-specific policy says the cap is expected and the scorer still receives a parseable prediction. For LongBench summarization and code rows, cap hits are inspected separately because truncation can hide task evidence or make throughput comparisons misleading. This validation layer is deliberately outside the router, so it can reject or quarantine rows without giving the method access to gold labels or terminal aggregate scores.

Backfill policy. Once raw rows land, numbers enter the manuscript through generated summaries and tables, not manual edits from terminal memory. The placeholder ledger names the owner artifact for each result token. Abstract, Results, Analysis, and captions should be regenerated from those artifacts so the paper cannot mix fresh tables with stale headline prose. Until that backfill happens, the scaffold remains a blocked draft even when it compiles cleanly.

Run integrity. At scaffold time, the repaired full-tail run is live and nonterminal, and it is writing partial `results.jsonl` rows for the missing-method matrix. The earlier all-source frontier directory remains incomplete, and no validated aggregate summary exists that combines it with the live tail. This draft therefore treats results as unavailable rather than substituting dry-run, smoke-only, failed-prefix, or nonterminal-prefix evidence.

Results

The final Results section will begin with a single cross-family matrix so reviewers can compare benchmark source, model, method, metric, mean KV budget, and quality in one place. At the time of this draft, that matrix is intentionally a placeholder because the live full-tail run is nonterminal and the earlier all-source frontier run has

Benchmark family	Source role	Planned rows	Main metric	Paper-facing status
LongBench	Real long-context tasks	At least 120 full rows	Macro score	P-LB
RULER	Controlled stress tasks	At least 80 full rows	Normalized score	P-RULER
Needle retrieval	Positional retrieval grid	At least 40 full rows	Retrieval score	P-NIAH
Systems subset	Timing and memory rows	Repeated trials	Memory / throughput	P-Sys

Table 2: Planned evidence matrix. The table records source families and owner placeholders only; completed result values must come from raw run artifacts.

not produced a validated aggregate. The headline comparison against the best nontrivial baseline frontier is `TBD_COMPARED_BASELINE_WIN_LOSS`.

The expected interpretation will be conservative. If Risk-routed KV improves quality but uses a larger average budget than the fixed frontier predicts, the result will be framed as a reliability-cost trade-off. If it improves quality at matched or interpolated mean KV budget, the result can support a frontier claim. If it does not improve either quality or systems cost, the method claim should be rejected while retaining the diagnostic analysis. The Results section will also report adverse comparisons directly. A strong fixed-budget baseline may dominate the router on some benchmark families; a systems table may show that routing overhead erases nominal KV savings; or the full cache may remain the only acceptable setting for tasks with fragile evidence spans. Those outcomes are not paper failures if they are measured cleanly. They determine whether the contribution is a method improvement, a deployment boundary, or an accounting and failure-analysis study.

Analysis and Ablation

The ablation plan separates task-family priors from row-level disagreement. The first ablation disables disagreement and tests whether task family alone is enough. The second uses disagreement without the task-family prior. The third compares against a fixed matched-budget policy on the same rows. The owner placeholder for the disagreement contribution is `TBD_ABLATION_DISAGREEMENT_DELTA`.

The ablation interpretation should distinguish signal quality from budget availability. If removing disagreement barely changes the frontier, then the task-family prior may be doing all useful work and the disagreement score should be demoted. If disagreement-only routing spends more KV without improving quality, then the signal may detect instability without identifying useful evidence. If both routed variants trail a fixed matched-budget policy, then heterogeneous routing is not justified by the current evidence. These cases require different revisions to the final thesis, so the ablation table should preserve row-level reason codes rather than only reporting aggregate scores.

The systems analysis must report both memory and speed. KV compression can reduce retained cache bytes while adding overhead in scoring, routing, or press application. For that reason, the final paper will report peak memory, prefill throughput, decode throughput, and end-to-end latency rather than only nominal retained-token percentages. The systems owner placeholders are `TBD_PEAK_MEMORY_REDUCTION`,

`TBD_DECODE_THROUGHPUT_GAIN`, and `TBD_SYSTEMS_LATENCY`. Systems rows should be interpreted on same-hardware measurements only. Cross-hardware comparisons are useful for debugging but cannot support a wall-clock frontier claim. For this reason, the systems subset must state whether memory and throughput were measured on the same GPU class and software path as the quality runs. If capacity constraints force a smaller systems subset, the final paper should say so and keep the memory and throughput claims scoped to that subset.

The statistical analysis should use paired row-level comparisons because all methods must run on the same selected benchmark rows within each source family. Bootstrap confidence intervals are planned for macro scores and frontier area. The owner placeholders are `TBD_SIGNIFICANCE_RESULT` and `TBD_CI_FRONTIER_AREA`.

Failure Cases

The failure analysis will use saved raw rows only. Expected categories include: aggressive compression drops an isolated answer span; escalation spends more KV but the model still answers from parametric knowledge or distractors; press disagreement is high but not predictive of answer failure; and a task-family prior over-escalates easy rows. The current failure-case rate is `TBD_FAILURE_CASE_RATE`. No qualitative example is included in this scaffold because non-terminal prefix rows have not been curated through a completed failure-taxonomy artifact.

The planned taxonomy separates recoverable and unrecoverable failures. Recoverable failures are rows where a safer budget or a different press preserves evidence that the aggressive default lost. Unrecoverable failures are rows where full cache also fails, the answer is ambiguous under the benchmark scorer, or the model ignores the relevant span despite retaining it. This distinction prevents the method from being blamed for all long-context errors. It also prevents the router from receiving credit for rows where the model would have succeeded at any budget.

A second axis is whether the risk signal fires for the right reason. Useful true positives are rows where the router escalates and the preserved evidence changes the answer. Costly false positives are easy rows that escalate without measurable quality benefit. False negatives are the most important safety failures: the router remains aggressive, deletes needed evidence, and loses quality relative to safer baselines. The final failure table should show this structure once raw rows exist, with examples drawn only from saved generations and scores.

Family	Model	Method	Mean KV budget	Metric	Result owner
LongBench	Qwen2.5-7B	Full cache	100%	Macro	P-Full
LongBench	Qwen2.5-7B	SnapKV frontier	25/50/75%	Macro	P-LB
LongBench	Qwen2.5-7B	Risk-routed KV	Adaptive	Macro	P-Main
RULER	Qwen2.5-7B	Baseline frontier	25/50/100%	Score	P-RULER
NIAH	Qwen2.5-7B	Risk-routed KV	Adaptive	Retrieval	P-NIAH
Systems	Qwen2.5-7B	Frontier subset	Adaptive	Memory/speed	P-Mem, P-Speed

Table 3: Main results matrix placeholder. Each result token is owned by a future run artifact listed in `paper/RESULT_PLACEHOLDERS.md`.

Conclusion

This scaffold defines a training-free KV-cache compression paper around a narrow, testable claim: risk-aware budget escalation should be evaluated as a quality-versus-KV-budget frontier, not as a universal same-budget salience score. The local evidence currently supports the research direction, benchmark plan, method accounting, and run-integrity blockers, but not a final result. The final conclusion must be regenerated after `TBD_MAIN_FRONTIER_DELTA`, `TBD_MATCHED_BUDGET_QUALITY`, and the systems placeholders are backed by completed run artifacts.

References

An, Y.; Lu, C.; Zhu, K.; Yu, T.; Zhao, C.; Wu, H.; et al. 2026. ReST-KV: Robust KV Cache Eviction with Layer-wise Output Reconstruction and Spatial-Temporal Smoothing. arXiv:2605.08840.

Bai, Y.; Lv, X.; Zhang, J.; Lyu, H.; Tang, J.; Huang, Z.; et al. 2024. LongBench: A Bilingual, Multitask Benchmark for Long Context Understanding. arXiv:2308.14508.

Behnam, P.; Fu, Y.; Zhao, R.; Tsai, P.-A.; Yu, Z.; and Tumanov, A. 2025. RocketKV: Accelerating Long-Context LLM Inference via Two-Stage KV Cache Compression. arXiv:2502.14051.

Bui, N.; Nguyen, H. T.; Cohan, A.; and Ying, R. 2026. Make Each Token Count: Towards Improving Long-Context Performance with KV Cache Eviction. arXiv:2605.09649.

Cai, Z.; Zhang, Y.; Gao, B.; Liu, Y.; Li, Y.; Liu, T.; et al. 2024. PyramidKV: Dynamic KV Cache Compression based on Pyramidal Information Funneling. arXiv:2406.02069.

Chen, A.; Geh, R.; Grover, A.; Van den Broeck, G.; and Israel, D. 2025. The Pitfalls of KV Cache Compression. arXiv:2510.00231.

Dao, T.; Fu, D. Y.; Ermon, S.; Rudra, A.; and Re, C. 2022. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. arXiv:2205.14135.

Feng, Y.; Lv, J.; Cao, Y.; Xie, X.; and Zhou, S. K. 2024. AdaKV: Optimizing KV Cache Eviction by Adaptive Budget Allocation for Efficient LLM Inference. arXiv:2407.11550.

Hsieh, C.-P.; Sun, S.; Krizan, S.; Acharya, S.; Rekesh, D.; Jia, F.; et al. 2024. RULER: What’s the Real Context Size of Your Long-Context Language Models? arXiv:2404.06654.

Kwon, W.; Li, Z.; Zhuang, S.; Sheng, Y.; Zheng, L.; Yu, C. H.; et al. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. arXiv:2309.06180.

Li, Y.; Huang, Y.; Yang, B.; Venkitesh, B.; Locatelli, A.; Ye, H.; et al. 2024. SnapKV: LLM Knows What You are Looking for Before Generation. arXiv:2404.14469.

Liu, X.; Tang, Z.; Dong, P.; Li, Z.; Liu, Y.; Li, B.; et al. 2025. ChunkKV: Semantic-Preserving KV Cache Compression for Efficient Long-Context LLM Inference. arXiv:2502.00299.

Liu, Z.; Yuan, J.; Jin, H.; Zhong, S.; Xu, Z.; Braverman, V.; et al. 2024. KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache. arXiv:2402.02750.

Patel, I.; and Joshi, I. 2026. PolyKV: A Shared Asymmetrically-Compressed KV Cache Pool for Multi-Agent LLM Inference. arXiv:2604.24971.

Shazeer, N. 2019. Fast Transformer Decoding: One Write-Head is All You Need. arXiv:1911.02150.

Vaswani, A.; Shazeer, N.; Parmar, N.; Uszkoreit, J.; Jones, L.; Gomez, A. N.; Kaiser, L.; and Polosukhin, I. 2017. Attention Is All You Need. arXiv:1706.03762.

Wang, G.; Upasani, S.; Wu, C.; Gandhi, D.; Li, J.; Hu, C.; et al. 2025. LLMs Know What to Drop: Self-Attention Guided KV Cache Eviction for Efficient Long-Context Inference. arXiv:2503.08879.

Wu, S.; Lin, H.; Chen, S.; Lu, T.; Zeng, Y.; Zhan, S.; et al. 2026. SemantiCache: Efficient KV Cache Compression via Semantic Chunking and Clustered Merging. arXiv:2603.14303.

Xiao, G.; Tian, Y.; Chen, B.; Han, S.; and Lewis, M. 2023. Efficient Streaming Language Models with Attention Sinks. arXiv:2309.17453.

Zhang, Z.; Sheng, Y.; Zhou, T.; Chen, T.; Zheng, L.; Cai, R.; et al. 2023. H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models. arXiv:2306.14048.

Reproducibility Checklist

Code and framework. The planned implementation uses project code over the maintained `kvpress` framework, with method provenance recorded in `experiments/KVPRESS_METHOD_MANIFEST.json`.

Data and benchmarks. Benchmark source URLs, local clones, task files, row hashes, and prepared split counts are recorded in `experiments/BENCHMARK_PROVENANCE.json`. Prepared rows are not counted as paper-facing evidence until corresponding raw result rows exist.

Compute and model. The planned model is Qwen2.5-7B-Instruct on a single H100-class GPU. Local CUDA ordinals, cache paths, and run-supervision internals are excluded from paper-facing claims but preserved in project manifests and `research/GROUND_TRUTH.md`.

Metrics and uncertainty. LongBench, RULER, NIAH, memory, throughput, paired comparisons, and bootstrap intervals are all placeholders pending completed raw rows.

Current blockers. The draft is not submission-ready. The full-tail run is live and nonterminal, partial rows are not final evidence, and the validated aggregate, full baseline matrix, ablations, systems measurements, and failure taxonomy remain missing.